

AWS N Git

AWS for software developer important concepts which should be know and tools
Interview Questions asked regrading AWS

For a **Software Developer (Java/Spring Boot/Microservices)** role, interviewers usually don't expect you to be an AWS Architect, but they do expect you to understand how applications are deployed, scaled, secured, and monitored on AWS.

AWS Roadmap for Software Developers

1. What is AWS?

AWS (Amazon Web Services) is a cloud platform that provides servers, databases, storage, networking, security, AI services, and much more on a pay-as-you-go model.

Instead of buying physical servers:

Traditional:
Application → Physical Server → Data Center

AWS:
Application → AWS Cloud → Pay only for usage

Benefits:

- No hardware management
- Auto scaling
- High availability
- Disaster recovery
- Security
- Global infrastructure

Most Important AWS Services for Developers

1. EC2 (Elastic Compute Cloud)

Virtual machine in AWS.

Example:

Spring Boot Application
↓
Deploy on EC2

↓
Accessible through Internet

Common interview question:

What is EC2?

A virtual server in AWS used to host applications.

EC2 Instance Types

Type	Purpose
T2/T3	General purpose
M5	Balanced
C5	Compute intensive
R5	Memory intensive

Example:

```
t3.micro
1 vCPU
1 GB RAM
```

Free tier eligible.

2. S3 (Simple Storage Service)

Object storage service.

Used for:

- Images
- Videos
- PDFs
- Backups
- Logs

Example:

```
User uploads profile photo
      ↓
Stored in S3
      ↓
URL stored in database
```

Features

- Unlimited storage

- High durability
- Versioning
- Encryption

Interview Question:

Difference between S3 and EBS?

S3	EBS
Object storage	Block storage
Stores files	Attached to EC2
Highly scalable	Single server storage

3. RDS (Relational Database Service)

Managed database service.

Supports:

- MySQL
- PostgreSQL
- Oracle
- SQL Server

Example:

```
Spring Boot
  ↓
RDS MySQL
```

AWS handles:

- Backup
- Replication
- Patching

Interview Question:

Why RDS over self-managed database?

- Automatic backups
- High availability
- Easy scaling
- Less maintenance

4. DynamoDB

NoSQL database.

Stores:

Key → Value

Example:

```
JSON
{
  "customerId": "1001",
  "name": "Lawrence"
}
```

Advantages:

- Serverless
- Auto scaling
- Millisecond response

Interview Question:

DynamoDB vs RDS

DynamoDB	RDS
NoSQL	SQL
Flexible schema	Fixed schema
Faster scaling	Complex joins

5. Lambda

Serverless computing.

No server management required.

Example:

```
File uploaded to S3
↓
Lambda Triggered
↓
Generate Thumbnail
```

Interview Question:

What is Lambda?

Runs code without provisioning servers.

Pay only for execution time.

6. API Gateway

Used to expose APIs.

```

Client
  ↓
API Gateway
  ↓
Lambda
  
```

Benefits:

- Authentication
- Rate limiting
- Monitoring

7. Elastic Beanstalk

PaaS Service.

Deploy application without managing infrastructure.

```

Spring Boot Jar
  ↓
Elastic Beanstalk
  ↓
AWS creates:
EC2
Load Balancer
Auto Scaling
  
```

Interview Question:

Difference between EC2 and Elastic Beanstalk?

EC2	Elastic Beanstalk
Manage everything	AWS manages infrastructure
More control	Easier deployment

8. ECS & EKS (Containers)

ECS

AWS Container Service.

Deploy Docker containers.

```

Docker Image
  ↓
ECS
  
```

EKS

Managed Kubernetes.

```

Docker
  ↓
Kubernetes
  ↓
EKS
  
```

Interview Question:

ECS vs EKS

ECS	EKS
AWS specific	Kubernetes
Easier	More flexible
Less learning	Industry standard

9. VPC (Virtual Private Cloud)

Private network in AWS.

Example:

```

VPC
├── Public Subnet
│   └── Load Balancer
└── Private Subnet
    └── Database
  
```

Interview Question:

Why VPC?

Provides network isolation and security.

10. Security Group

AWS Firewall.

Controls:

Inbound Traffic
Outbound Traffic

Example:

Port 22 → SSH
Port 80 → HTTP
Port 443 → HTTPS

Interview Question:

Security Group vs NACL

Security Group	NACL
Instance level	Subnet level
Stateful	Stateless

11. Load Balancer

Distributes traffic.

Users
↓
Load Balancer
↓
Server1
Server2
Server3

Types:

- Application Load Balancer (ALB)
- Network Load Balancer (NLB)

12. Auto Scaling

Automatically adds/removes servers.

Example:

100 Users → 2 Servers

5000 Users
↓
Auto Scaling
↓
10 Servers

13. CloudWatch

Monitoring service.

Tracks:

- CPU
- Memory
- Logs
- Errors

Example:

```
CPU > 80%
  ↓
Alarm Triggered
```

Interview Question:

How do you monitor a Spring Boot application on AWS?

- CloudWatch Logs
- CloudWatch Metrics
- Alarms

14. IAM (Identity and Access Management)

Security service.

Controls:

- Users
- Roles
- Permissions

Example:

```
Developer
  ↓
Read-only S3 Access
```

Interview Question:

IAM User vs IAM Role

User	Role
Permanent	Temporary
Login credentials	Assumed by services

Example:


```

EC2
↓
IAM Role
↓
Access S3

```

No credentials needed in code.

15. Route 53

DNS Service.

Example:

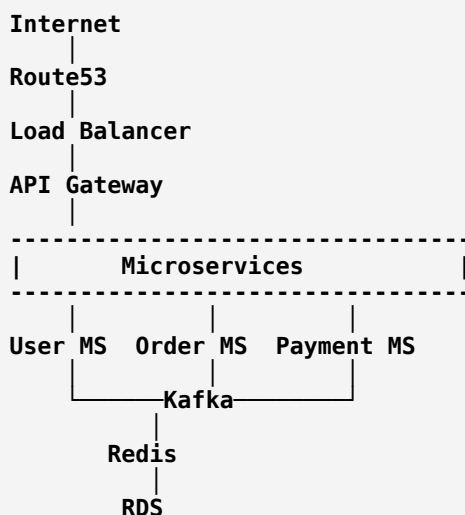
```

api.company.com
↓
AWS Route53
↓
Load Balancer

```

AWS Architecture for Microservices

Typical Interview Architecture:



AWS + Spring Boot Interview Scenario

How do you deploy Spring Boot application on AWS?

Answer:

1. Build Jar
- `mvn clean package`
2. Launch EC2
3. Install Java
4. Upload Jar

5. Run

```
java -jar app.jar
```

6. Open Security Group Port

Production:

```
Spring Boot
  ↓
Docker
  ↓
ECS/EKS
  ↓
ALB
  ↓
Auto Scaling
```

Frequently Asked AWS Interview Questions

Beginner

1. What is AWS?
2. What is EC2?
3. What is S3?
4. What is RDS?
5. What is VPC?
6. What is IAM?
7. What is Route53?
8. What is Lambda?
9. What is Auto Scaling?
10. What is CloudWatch?

Intermediate

1. Difference between EC2 and Lambda?
2. Difference between S3 and EBS?
3. Difference between Security Group and NACL?
4. Difference between ECS and EKS?
5. Difference between RDS and DynamoDB?
6. What is an Availability Zone?
7. What is an AMI?
8. How does Auto Scaling work?
9. How do you secure an EC2 instance?
10. How would you deploy a Spring Boot microservice?

Scenario-Based Questions

Application is slow. How do you troubleshoot?

Check:

- CloudWatch Metrics
- CPU usage
- Memory usage
- Database latency
- Network latency
- Logs

EC2 crashed. What happens?

If:

EC2 + Auto Scaling Group

AWS automatically launches a new instance.

High traffic suddenly comes.

Use:

**Load Balancer
+
Auto Scaling**

Secure access to S3 from EC2?

Use:

IAM Role

Avoid:

**Java
Hardcoded Access Keys**

AWS Services Most Asked in Java/Spring Boot Interviews

Focus heavily on:

1. EC2
2. S3

3. RDS
4. IAM
5. VPC
6. Security Groups
7. Load Balancer
8. Auto Scaling
9. CloudWatch
10. Lambda
11. ECS
12. Docker + AWS
13. Route53
14. API Gateway
15. Redis (via AWS ElastiCache)

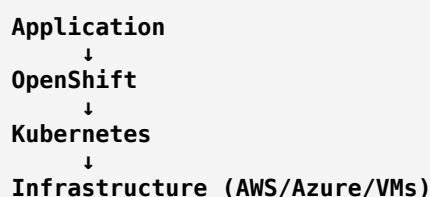
If you're preparing for **2-5 years Java Spring Boot/Microservices interviews**, mastering **EC2, S3, RDS, IAM, VPC, Load Balancer, Auto Scaling, CloudWatch, ECS, Lambda, and API Gateway** will cover about **80-90% of AWS questions** commonly asked.

Openshift concepts need to know for software developer
Openshift interview Questions asked

OpenShift for Software Developers (Spring Boot/Microservices)

Many companies use **OpenShift** instead of managing Kubernetes directly because it provides an enterprise platform on top of Kubernetes with security, CI/CD, monitoring, and deployment features.

Think of it as:



For a Software Developer, you don't need deep cluster administration knowledge, but you should understand deployment, scaling, networking, containers, and troubleshooting.

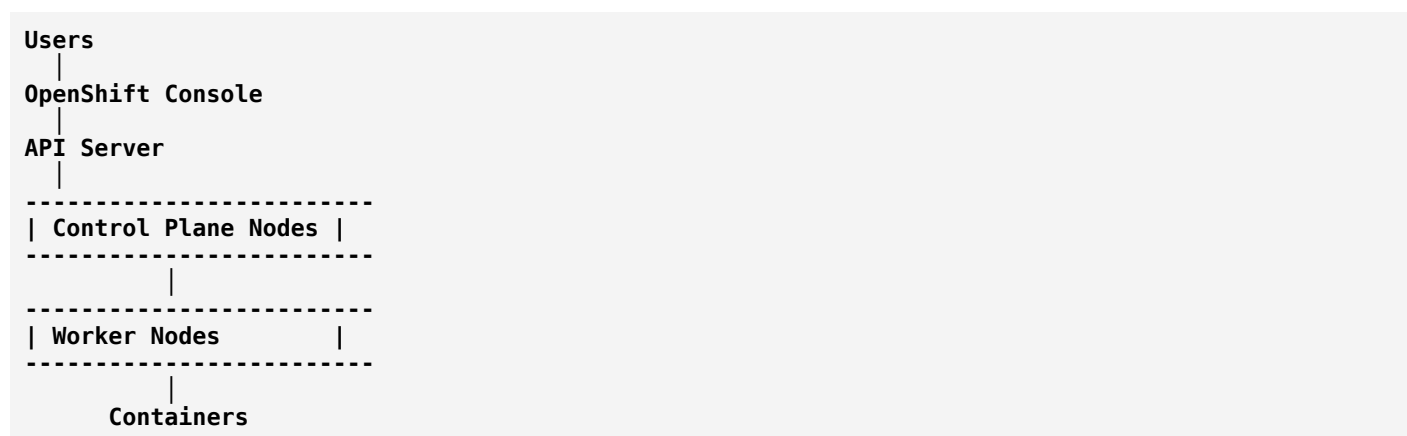
What is OpenShift?

OpenShift is a Kubernetes-based container platform developed by [Red Hat](#) .

It provides:

- Kubernetes orchestration
- Container deployment
- CI/CD integration
- Security
- Monitoring
- Logging
- Scaling

OpenShift Architecture



Components

1. Master/Control Plane
2. Worker Nodes
3. Pods
4. Services
5. Routes
6. DeploymentConfig/Deployment
7. Image Registry

OpenShift vs Kubernetes

Kubernetes	OpenShift
Open Source	Enterprise Platform
Manual setup	Ready-to-use platform
Basic security	Enhanced security
No built-in CI/CD	CI/CD integration
Complex	Easier UI

Interview Question:

Why OpenShift when Kubernetes already exists?

OpenShift provides:

- Better security
- Developer-friendly UI
- Built-in registry
- CI/CD pipelines
- Enterprise support

Container Fundamentals

OpenShift runs containers.

Example:

```
Spring Boot App
  ↓
Docker Image
  ↓
OpenShift
```

Developer responsibility:

```
Code
  ↓
Dockerfile
  ↓
Container Image
  ↓
Deploy
```

Pods

Smallest deployable unit in Kubernetes/OpenShift.

```
Pod
├─ Spring Boot Container
```

Multiple containers can exist in one pod.

```
Pod
├─ App Container
├─ Logging Container
```

Interview Question:

What is a Pod?

A Pod is the smallest deployable unit that contains one or more containers sharing network and storage.

Deployment

Deployment manages pods.

```
Deployment
  ↓
3 Replicas
  ↓
Pod1
Pod2
Pod3
```

Benefits:

- Auto healing
- Rolling updates
- Scaling

ReplicaSet

Maintains desired number of pods.

Example:

```
Desired Pods = 3
Pod crashes
  ↓
ReplicaSet creates new Pod
```

Interview Question:

What happens when a Pod crashes?

ReplicaSet automatically creates a new Pod.

Services

Pods are temporary.

Service provides a stable endpoint.

```
Client
  ↓
Service
  ↓
Pods
```

Without Service:

```
Pod IP changes
```

With Service:

Stable URL

Routes (OpenShift Specific)

Routes expose applications outside the cluster.

```

Internet
  ↓
Route
  ↓
Service
  ↓
Pods
  
```

Example:

`https://customer-app.company.com`

Interview Question:

Difference between Service and Route?

Service	Route
Internal communication	External access
Cluster only	Internet accessible

ConfigMaps

Store configuration.

Example:

```

YAML
DB_URL=jdbc:mysql://...
  
```

Avoid hardcoding.

```

ConfigMap
  ↓
Pod
  
```

Interview Question:

Why ConfigMap?

To externalize configuration from application code.

Secrets

Store sensitive information.

Examples:

- Passwords
- API Keys
- Tokens

Secret
↓
Application

Interview Question:

Difference between ConfigMap and Secret?

ConfigMap	Secret
Normal configuration	Sensitive data
Plain text	Encoded & protected

Persistent Volumes (PV)

Storage for containers.

Without PV:

Pod deleted
↓
Data lost

With PV:

Pod deleted
↓
Data remains

Used for:

- Logs
- Uploads
- Databases

Scaling

Manual Scaling

```
Bash
oc scale deployment customer-service --replicas=5
```

Result:

```
Pod1
Pod2
Pod3
Pod4
Pod5
```

Auto Scaling (HPA)

Horizontal Pod Autoscaler.

```
CPU > 80%
  ↓
3 Pods
  ↓
8 Pods
```

Interview Question:

Difference between Vertical and Horizontal Scaling?

Vertical	Horizontal
Increase CPU/RAM	Increase pods
One server	Multiple servers

Rolling Deployment

Zero downtime deployment.

```
Old Pod
Old Pod
Old Pod

↓

New Pod
Old Pod
Old Pod

↓

New Pod
New Pod
Old Pod

↓

New Pod
New Pod
New Pod
```

Interview Question:

How is downtime avoided during deployment?

Using rolling updates.

Health Checks

Liveness Probe

Checks:

Is application alive?

If failed:

Restart container

Readiness Probe

Checks:

Can application accept traffic?

If failed:

Remove from Load Balancer

Interview Question:

Liveness vs Readiness Probe?

Liveness	Readiness
Restart container	Stop traffic
App health	App availability

OpenShift Build Concepts

BuildConfig

Automates image creation.

Git Repository
↓
BuildConfig

↓
Docker Image

Image Stream

Tracks container images.

Image Stream
 ↓
Version Tracking

Benefits:

- Easier updates
- Rollbacks

Interview Question:

What is Image Stream?

An OpenShift object that tracks image versions.

CI/CD Pipeline

Typical Flow:

Developer
 ↓
Git Push
 ↓
Jenkins/Tekton
 ↓
Build Image
 ↓
OpenShift Deployment

OpenShift Networking

Browser
 ↓
Route
 ↓
Service
 ↓
Pod

Important interview question.

Explain request flow in OpenShift.

Answer:

User Request
 ↓

```
Route
  ↓
Service
  ↓
Pod
  ↓
Application Response
```

OpenShift Security

One major difference from Kubernetes.

Security Context Constraints (SCC)

Controls:

- Privileges
- User permissions
- Container execution

OpenShift generally does NOT allow:

```
Run as Root
```

Interview Question:

Why does OpenShift not allow root containers?

For better security and isolation.

OpenShift Commands

Login

```
Bash
oc login
```

Projects

```
Bash
oc get projects
```

Pods

```
Bash
oc get pods
```

Services

```
Bash
oc get svc
```

Deployments

```
Bash
oc get deployments
```

Logs

```
Bash
oc logs pod-name
```

Describe Pod

```
Bash
oc describe pod pod-name
```

Very common interview question.

Troubleshooting Scenarios

Scenario 1

Application not accessible.

Check:

```
Route
↓
Service
↓
Pod
```

Verify all exist.

Scenario 2

Pod continuously restarting.

Check:

```
Bash
oc logs pod-name
```

Possible reasons:

- Application crash

- Memory issue
 - DB connectivity issue
-

Scenario 3

Application slow.

Check:

CPU
Memory
Network
Database

Use monitoring tools.

Frequently Asked OpenShift Interview Questions

Basic

1. What is OpenShift?
 2. Difference between OpenShift and Kubernetes?
 3. What is a Pod?
 4. What is a Service?
 5. What is a Route?
 6. What is a Deployment?
 7. What is ReplicaSet?
 8. What is ConfigMap?
 9. What is Secret?
 10. What is Image Stream?
-

Intermediate

1. Explain OpenShift architecture.
2. Explain Route → Service → Pod flow.
3. What happens when a Pod crashes?
4. How does scaling work?
5. How does rolling deployment work?
6. What is BuildConfig?

7. What is SCC?
 8. How do ConfigMaps work?
 9. How do Secrets work?
 10. Difference between Deployment and DeploymentConfig?
-

Scenario-Based Questions

Your pod is in CrashLoopBackOff state. What will you do?

Answer:

1. Check pod logs
2. Check environment variables
3. Check DB connectivity
4. Check memory limits
5. Check application startup errors

Application is deployed but URL not working.

Check:

Route
Service
Pod
Port Mapping
Network Policies

One pod is receiving high traffic.

Solution:

Increase Replicas
Enable HPA
Use Load Balancing

Most Important OpenShift Topics for 2-5 Years Java/Spring Boot Developers

Focus on:

1. Containers & Docker
2. Pods
3. Deployments
4. ReplicaSets
5. Services
6. Routes
7. ConfigMaps

8. Secrets
9. Persistent Volumes
10. Auto Scaling
11. Rolling Updates
12. Liveness Probe
13. Readiness Probe
14. BuildConfig
15. Image Streams
16. CI/CD Pipelines
17. OpenShift Security (SCC)
18. Troubleshooting Pods
19. Route → Service → Pod request flow
20. OpenShift vs Kubernetes

These topics cover the vast majority of OpenShift questions asked in Java Spring Boot, Microservices, and DevOps-focused software developer interviews.

Linux for software developer
Linux commands which should be known

Linux for Software Developers

Most Java/Spring Boot/Microservices applications run on Linux servers. In interviews, you are not expected to be a Linux Administrator, but you should be comfortable with navigation, process management, logs, networking, permissions, and troubleshooting.

Why Developers Need Linux?

Typical Production Environment:

```

Developer
  ↓
Git
  ↓
Jenkins
  ↓
Docker
  ↓
Linux Server
  ↓
Spring Boot Application
  
```

Common Tasks:

- Check logs

- Start/Stop applications
- Monitor CPU/Memory
- Check ports
- Verify database connectivity
- Debug production issues

Linux File System Basics

```

/
├── bin
├── boot
├── dev
├── etc
├── home
├── opt
├── tmp
├── usr
└── var
  
```

Important Directories:

Directory	Purpose
/home	User files
/etc	Configuration files
/var/log	Application logs
/tmp	Temporary files
/opt	Installed applications
/usr	System programs

Interview Question:

What is the root directory?

The top-most directory represented by /.

Navigation Commands

Current Directory

```

Bash
pwd
  
```

Output:

```

/home/lawrence/projects
  
```

List Files

```
Bash  
ls
```

Detailed:

```
Bash  
ls -l
```

Hidden files:

```
Bash  
ls -a
```

Change Directory

```
Bash  
cd foldername
```

Go back:

```
Bash  
cd ..
```

Go home:

```
Bash  
cd ~
```

File Operations

Create File

```
Bash  
touch test.txt
```

Create Directory

```
Bash  
mkdir project
```

Copy

```
Bash
cp source.txt destination.txt
```

Copy folder:

```
Bash
cp -r sourceFolder targetFolder
```

Move/Rename

```
Bash
mv old.txt new.txt
```

Delete File

```
Bash
rm file.txt
```

Delete folder:

```
Bash
rm -r folder
```

Force delete:

```
Bash
rm -rf folder
```

Interview Question:

Difference between `rm -r` and `rm -rf`?

- `-r` → Recursive
- `-f` → Force delete without confirmation

Viewing File Content

Display File

```
Bash
cat file.txt
```

First 10 Lines

```
Bash
head file.txt
```

Last 10 Lines

```
Bash
tail file.txt
```

Real-Time Logs

Very common in production:

```
Bash
tail -f application.log
```

Example:

```
Bash
tail -f catalina.out
```

Interview Question:

How do you monitor application logs in real time?

```
Bash
tail -f logfile.log
```

Searching

Find Text

```
Bash
grep "ERROR" app.log
```

Case insensitive:

```
Bash
grep -i error app.log
```

Count occurrences:

```
Bash
grep -c ERROR app.log
```

Find Files

```
Bash
find . -name "*.log"
```

Example:

```
Bash
find . -name "*.jar"
```

Permissions

Linux permissions:

```
-rwxr-xr-x
```

Meaning:

Owner	Group	Others
rwX	r-x	r-x

Symbol	Meaning
r	Read
w	Write
x	Execute

Change Permissions

```
Bash
chmod 755 app.sh
```

Interview Question:

What does chmod 755 mean?

```
Owner = rwx
Group = r-x
Others = r-x
```

Change Owner

```
Bash
chown user file.txt
```

Process Management

Very important for interviews.

Show Running Processes

```
Bash  
ps -ef
```

Find Java Process

```
Bash  
ps -ef | grep java
```

Example Output:

```
12345 java -jar customer-service.jar
```

Interview Question:

How do you check if your Spring Boot application is running?

```
Bash  
ps -ef | grep java
```

Kill Process

```
Bash  
kill PID
```

Force Kill:

```
Bash  
kill -9 PID
```

Example:

```
Bash  
kill -9 12345
```

Interview Question:

Difference between kill and kill -9?

kill	kill -9
Graceful stop	Force stop
Cleanup possible	Immediate termination

Memory Monitoring

Free Memory

```
Bash
free -m
```

Output:

```
Total
Used
Free
```

Detailed Memory Usage

```
Bash
top
```

or

```
Bash
htop
```

Interview Question:

How do you check memory usage?

```
Bash
free -m
top
```

CPU Monitoring

```
Bash
top
```

Shows:

- CPU
- Memory

- Running Processes

Disk Space

Check Disk

```
Bash
df -h
```

Example:

```
Filesystem Size Used Avail
```

Folder Size

```
Bash
du -sh foldername
```

Interview Question:

Disk full in production. How will you investigate?

```
Bash
df -h
du -sh *
```

Check logs:

```
Bash
/var/log
```

Networking Commands

Most Important Topic

Check IP Address

```
Bash
ip addr
```

or

```
Bash
ifconfig
```

Ping

```
Bash
ping google.com
```

Checks connectivity.

Check Port

```
Bash
netstat -tulpn
```

or

```
Bash
ss -tulpn
```

Find Process Using Port

```
Bash
lsof -i :8080
```

Interview Question:

Port 8080 already in use. How do you find the process?

```
Bash
lsof -i :8080
```

Test Endpoint

```
Bash
curl http://localhost:8080/actuator/health
```

Very common for Spring Boot.

Environment Variables

View all:

```
Bash
env
```

Specific:

```
Bash
echo $JAVA_HOME
```

Set:

```
Bash
export JAVA_HOME=/usr/java
```

Interview Question:

What is JAVA_HOME?

Environment variable pointing to Java installation directory.

Package Management

Ubuntu:

```
Bash
apt install package
```

RedHat/CentOS:

```
Bash
yum install package
```

or

```
Bash
dnf install package
```

Compression

Zip:

```
Bash
zip file.zip file.txt
```

Unzip:

```
Bash
unzip file.zip
```

Tar:

```
Bash
```

```
tar -cvf backup.tar folder
```

Extract:

```
Bash  
tar -xvf backup.tar
```

Useful Developer Commands

Java Version

```
Bash  
java -version
```

Maven Version

```
Bash  
mvn -version
```

Git Version

```
Bash  
git --version
```

Check Open Ports

```
Bash  
netstat -an
```

Run Spring Boot Jar

```
Bash  
java -jar customer-service.jar
```

Background:

```
Bash  
nohup java -jar customer-service.jar &
```

Interview Question:

What does nohup do?

Runs process even after terminal closes.

Log Analysis Commands

Find ERROR logs:

```
Bash
grep ERROR app.log
```

Count errors:

```
Bash
grep ERROR app.log | wc -t
```

Latest errors:

```
Bash
tail -100 app.log
```

Most Asked Linux Interview Questions

Basic

1. What is Linux?
 2. Difference between Linux and Windows?
 3. What is root user?
 4. What is a shell?
 5. What is a process?
-

Commands

1. Difference between grep and find?
 2. Difference between cp and mv?
 3. Difference between kill and kill -9?
 4. Difference between top and ps?
 5. What does chmod 755 mean?
-

Developer Scenarios

Application not starting.

Check:

```
Bash
ps -ef | grep java
tail -f app.log
```

Application inaccessible.

Check:

```
Bash
netstat -tulpn
```

or

```
Bash
ss -tulpn
```

Verify port is listening.

High CPU.

Check:

```
Bash
top
```

Find offending process.

Memory issue.

Check:

```
Bash
free -m
top
```

Port already in use.

Find process:

```
Bash
lsof -i :8080
```

Kill process:

```
Bash
kill -9 PID
```

Linux Commands Every Java/Spring Boot Developer Should Know

Bash

```
pwd
ls -lt
cd
mkdir
touch
cp
mv
rm -rf
cat
head
tail -f
grep
find
chmod
chown
ps -ef
kill
kill -9
top
free -m
df -h
du -sh
ping
curl
netstat
ss
lsof
env
export
zip
unzip
tar
java -version
mvn -version
nohup
```

If you're preparing for **Java, Spring Boot, Microservices, OpenShift, AWS, or DevOps-related interviews**, focus especially on:

- grep
- tail -f
- ps -ef
- kill
- top
- free -m
- df -h
- curl
- lsof
- netstat/ss
- nohup

These are the commands developers use most often when troubleshooting production issues.

Firestore basic knowledge

Firebase Basic Knowledge for Software Developers

What is Firebase?

Firebase is a **Backend-as-a-Service (BaaS)** platform provided by [Firebase](#) and backed by [Google Cloud](#).

It provides ready-made backend services such as:

- Authentication
- Database
- File Storage
- Push Notifications
- Hosting
- Analytics
- Crash Reporting

Instead of building a backend from scratch:

```

Traditional
Mobile App
  ↓
Spring Boot API
  ↓
Database
    
```

With Firebase:

```

Mobile/Web App
  ↓
Firebase Services
  ↓
Database/Auth/Storage
    
```

Why Use Firebase?

Advantages:

- Fast development
- Serverless
- Real-time updates
- Easy authentication
- Push notifications
- Scalable

Commonly used for:

- Android Apps
- iOS Apps
- React Applications
- MVPs
- Chat Applications

Core Firebase Services

1. Firebase Authentication

Provides user login functionality.

Supports:

- Email/Password
- Google Login
- Facebook Login
- Apple Login
- Phone OTP Login

Example:

```

User Login
  ↓
Firebase Auth
  ↓
JWT Token Generated
  
```

Interview Question:

Why use Firebase Authentication?

It eliminates the need to build and maintain authentication infrastructure.

2. Cloud Firestore

Modern NoSQL database.

Structure:

```

Collection
  ↓
Document
  ↓
Fields
  
```

Example:

```
JSON
users
└─ user1
   {
     "name": "Lawrence",
     "city": "Mumbai"
   }
```

Unlike SQL:

```
SQL
Table -> Row
```

Firestore:

```
Collection -> Document
```

Features

- Real-time updates
- Auto scaling
- Offline support
- Serverless

3. Realtime Database

Older Firebase database.

```
JSON Tree Structure
```

Example:

```
JSON
{
  "users": {
    "1001": {
      "name": "Lawrence"
    }
  }
}
```

Firestore vs Realtime Database

Firestore	Realtime Database
Newer	Older
Better querying	Limited querying
More scalable	Less scalable
Preferred today	Legacy projects

Interview Question:

Which database should you choose?

For new applications, prefer **Cloud Firestore**.

4. Firebase Storage

Stores:

- Images
- Videos
- PDFs
- Documents

Example:

```

Upload Image
  ↓
Firebase Storage
  ↓
URL Generated
  ↓
Save URL in Database
  
```

Common use case:

Profile Pictures

5. Firebase Cloud Messaging (FCM)

Push Notification Service.

Example:

```

Backend
  ↓
FCM
  ↓
Android App
  
```

Notification:

"Your order has been delivered"

Interview Question:

What is FCM?

Firebase Cloud Messaging is used for sending push notifications to mobile and web applications.

6. Firebase Hosting

Used to host:

- Angular Apps
- React Apps
- Static Websites

Example:

```
React Application
  ↓
Firebase Hosting
```

Deployment:

```
Bash
firebase deploy
```

7. Firebase Analytics

Tracks:

- Users
- Screen Views
- Events
- Sessions

Example:

```
Button Click
  ↓
Analytics Event
```

Useful for product teams.

8. Crashlytics

Monitors application crashes.

Example:

```
App Crash
  ↓
Crashlytics
```

Detailed Error Report

Interview Question:

What is Crashlytics?

A Firebase service that captures crash reports and application errors.

How Firebase Works

```

Mobile App
  ↓
Firebase SDK
  ↓
Firebase Services
  ↓
Firestore
Auth
Storage
FCM
  
```

No server management required.

Firestore Authentication Flow

```

User Login
  ↓
Firebase Auth
  ↓
Token Generated
  ↓
Token Returned
  ↓
Protected API Access
  
```

Similar to JWT authentication.

Firestore Data Example

Collection:

```
customers
```

Document:

```

JSON
{
  "customerId": "1001",
  "name": "Lawrence",
  "email": "lawrence@gmail.com"
}
  
```

Query:

```
Java
db.collection("customers")
```

Firestore Security Rules

Very important topic.

Example:

```
JavaScript
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read, write: if request.auth != null;
    }
  }
}
```

Meaning:

```
Only authenticated users
can access data.
```

Interview Question:

Why are Firestore Security Rules important?

They control access to database and storage resources.

Firestore vs Spring Boot Backend

Firestore	Spring Boot
Serverless	Custom Backend
Faster setup	More flexibility
Managed services	Full control
Limited complex business logic	Complex logic supported
Less infrastructure	More infrastructure

When to Use Firestore

Good Choice:

- ✓ Chat Application
- ✓ Mobile Application
- ✓ MVP

- ✓ Startup Product
 - ✓ Real-time Applications
 - ✓ Push Notifications
-

Not Ideal:

- ✗ Complex Enterprise Systems
- ✗ Heavy Business Logic
- ✗ Complex Transactions
- ✗ Large Microservice Architectures

For enterprise banking, insurance, and large-scale Java microservices, Spring Boot + Database is usually preferred.

Firestore Interview Questions

Basic

What is Firebase?

A Backend-as-a-Service platform that provides authentication, databases, storage, notifications, analytics, and hosting.

What is Firestore?

A NoSQL cloud database for storing application data.

What is Firebase Authentication?

A managed authentication service supporting email/password, social login, and phone authentication.

What is Firebase Storage?

Cloud storage for images, videos, and files.

What is FCM?

Firebase Cloud Messaging used for push notifications.

Intermediate Questions

Difference between Firestore and Realtime Database?

Firestore	Realtime Database
Document-based	JSON Tree
Better scalability	Less scalable
Rich querying	Basic querying

Firestore vs AWS?

Firestore	AWS
Simpler	More powerful
Mobile focused	Enterprise focused
Less configuration	More flexibility

Firestore vs Spring Boot?

Firestore	Spring Boot
Serverless	Backend framework
Managed services	Full custom backend
Faster development	More control

Scenario-Based Questions

How would you implement login in a mobile application?

Use:

```

Firestore Authentication
  ↓
Email/Google Login
  ↓
JWT Token

```

How would you implement push notifications?

Use:

```

Firestore Cloud Messaging (FCM)

```

User uploads profile picture. How would you store it?

```

Upload
  ↓
Firestore Storage

```


↓
Store URL in Firestore

Most Important Firebase Topics for Interviews

1. Firebase Overview
2. Firebase Authentication
3. Cloud Firestore
4. Realtime Database
5. Firebase Storage
6. Firebase Cloud Messaging (FCM)
7. Firebase Hosting
8. Firebase Analytics
9. Crashlytics
10. Security Rules
11. Firestore vs Realtime Database
12. Firebase vs Spring Boot
13. Firebase vs AWS
14. Push Notifications
15. Real-Time Data Synchronization

For a Java/Spring Boot developer, interviewers usually ask Firebase only at a **basic-to-intermediate level**, focusing on Authentication, Firestore, FCM, and comparisons with traditional backend architectures.

Git/GitHub Gitlab basic commands and concepts

Git, GitHub & GitLab for Software Developers

This is one of the most important interview topics for Java/Spring Boot developers because almost every company uses Git-based version control.

What is Git?

Git is a distributed version control system used to track changes in source code.

Without Git:

```
Developer A
  ↓
code.zip
Developer B
  ↓
```

```
code-final.zip
Developer C
↓
code-final-final.zip
```

With Git:

```
Developer A
↓
Git Repository
↑
Developer B
↑
Developer C
```

Benefits:

- Version control
- Team collaboration
- Code history
- Branching
- Rollback capability

Git vs GitHub vs GitLab

Git

Version control tool.

Installed locally:

```
Bash
git --version
```

GitHub

Cloud platform for hosting Git repositories.

Features:

- Pull Requests
- Code Review
- CI/CD
- Issue Tracking

GitLab

Git repository platform similar to GitHub.

Features:

- Built-in CI/CD
- Security scanning
- DevOps tooling

Interview Question

Difference between Git and GitHub?

Git	GitHub
Version control tool	Repository hosting platform
Local machine	Cloud
Tracks code	Stores repositories

Git Workflow

```

Working Directory
  ↓
Staging Area
  ↓
Local Repository
  ↓
Remote Repository

```

Flow:

```

Bash
git add
git commit
git push

```

Important Git Terminologies

Repository (Repo)

A project managed by Git.

```

Bash
git init

```

Creates:

```

.git

```

directory.

Commit

A snapshot of code at a point in time.

```
Bash
git commit -m "Added login feature"
```

Interview Question:

What is a commit?

A snapshot of project changes saved in Git history.

Branch

Independent line of development.

```
main
├── feature/login
└── feature/payment
```

Benefits:

- Parallel development
- Isolation

Merge

Combines branches.

```
feature/login
    ↓
  Merge
    ↓
main
```

Most Used Git Commands

Initialize Repository

```
Bash
```

```
git init
```

Clone Repository

```
Bash
git clone https://github.com/project.git
```

Copies remote repository locally.

Check Status

```
Bash
git status
```

Very frequently used.

Shows:

```
Modified Files
Untracked Files
Branch
```

View Changes

```
Bash
git diff
```

Shows modified code.

Add Files

Single file:

```
Bash
git add file.java
```

All files:

```
Bash
git add .
```

Commit Changes

```
Bash
git commit -m "Added customer API"
```

View Commit History

```
Bash
git log
```

Short format:

```
Bash
git log --oneline
```

Remote Repository Commands

Add Remote

```
Bash
git remote add origin repository-url
```

Check remote:

```
Bash
git remote -v
```

Push Code

```
Bash
git push origin main
```

Example:

```
Bash
git push origin feature/login
```

Pull Latest Changes

```
Bash
git pull origin main
```

Very common.

Equivalent:

Fetch + Merge

Fetch Changes

```
Bash
git fetch
```

Downloads changes but doesn't merge.

Interview Question:

Difference between pull and fetch?

Pull	Fetch
Downloads + Merges	Downloads only
Updates branch	No merge

Branch Commands

List Branches

```
Bash
git branch
```

Create Branch

```
Bash
git branch feature-login
```

Switch Branch

```
Bash
git checkout feature-login
```

Modern:

```
Bash
git switch feature-login
```

Create and Switch

```
Bash
git checkout -b feature-login
```

Delete Branch

```
Bash
git branch -d feature-login
```

Merge Commands

Switch to main:

```
Bash
git checkout main
```

Merge:

```
Bash
git merge feature-login
```

Merge Conflict

Occurs when two developers modify same code.

Example:

Developer A:

```
Java
String name = "John";
```

Developer B:

```
Java
String name = "Lawrence";
```

Git:

```
CONFLICT
```

Resolution:

```
Choose correct code
Commit again
```

Interview Question:

What is a merge conflict?

Occurs when Git cannot automatically combine changes from multiple branches.

Stash

Temporarily save work.

Save:

```
Bash
git stash
```

Restore:

```
Bash
git stash pop
```

Scenario:

```
Working on Feature A
Need urgent bug fix
```

Use stash.

Interview Question:

Why use git stash?

Temporarily stores uncommitted changes.

Reset

Undo changes.

Soft Reset

```
Bash
git reset --soft HEAD~1
```

Undo commit but keep changes.

Hard Reset

```
Bash
git reset --hard HEAD~1
```

Delete commit and changes.

Be careful.

Interview Question:

Difference between soft and hard reset?

Soft	Hard
Keeps files	Removes files
Safe	Destructive

Revert

Safe undo.

```
Bash
git revert commit-id
```

Creates a new commit that reverses changes.

Preferred in production.

Cherry Pick

Copy commit from one branch to another.

```
Bash
git cherry-pick commit-id
```

Scenario:

```
Need one bug fix
Not entire branch
```

Tags

Mark releases.

```
Bash
git tag v1.0
```

Example:

```
v1.0
v2.0
v3.0
```

Pull Request (PR)

GitHub/GitLab workflow.

```
Feature Branch
  ↓
Pull Request
  ↓
Code Review
  ↓
Merge
```

Interview Question:

What is a Pull Request?

A request to merge code into another branch after review.

GitLab CI/CD

Flow:

```
Code Push
  ↓
GitLab Pipeline
  ↓
Build
  ↓
Test
  ↓
Deploy
```

Pipeline file:

```
.gitlab-ci.yml
```

Common Interview Scenarios

Scenario 1

Accidentally committed wrong code.

Solution:

```
Bash
git revert commit-id
```

or

```
Bash
git reset
```

Scenario 2

Need latest code.

```
Bash
git pull origin main
```

Scenario 3

Need temporary save.

```
Bash
git stash
```

Scenario 4

Bring one commit from another branch.

```
Bash
git cherry-pick commit-id
```

Scenario 5

View changed files.

```
Bash
git status
git diff
```

Most Asked Git Interview Questions

Basic

1. What is Git?
2. What is GitHub?
3. What is GitLab?
4. What is a repository?
5. What is a commit?
6. What is a branch?
7. What is merge?

Intermediate

1. Difference between pull and fetch?
 2. Difference between merge and rebase?
 3. What is stash?
 4. What is cherry-pick?
 5. What is revert?
 6. What is reset?
 7. What is a Pull Request?
-

Advanced

1. How do you resolve merge conflicts?
 2. What is Git rebase?
 3. Difference between revert and reset?
 4. Explain Git workflow in your project.
 5. How do you recover deleted commits?
 6. What happens during pull?
 7. Explain GitLab CI/CD pipeline.
-

Commands Every Software Developer Must Know

Bash

```
git init
git clone
git status
git add .
git commit -m "message"
git log
git diff
git branch
git checkout
git switch
git checkout -b branch-name
git merge
git pull
git fetch
git push
git stash
git stash pop
git reset
git revert
git cherry-pick
git tag
git remote -v
```

Questions Frequently Asked in Java/Spring Boot Interviews

1. Difference between Git and GitHub?
2. What happens internally during `git pull`?
3. Difference between Merge and Rebase?
4. How do you resolve merge conflicts?
5. What is Stash and when have you used it?
6. Difference between Reset and Revert?
7. Explain your branching strategy.
8. What is a Pull Request?
9. What is GitLab CI/CD?
10. How do developers collaborate using Git?

For 2-5 years experience, you should be especially comfortable with:

- Branching
- Merge vs Rebase
- Pull Requests
- Stash
- Cherry Pick
- Revert vs Reset
- GitLab Pipelines
- Conflict Resolution

These are asked far more often than obscure Git commands.

SonarQube and JUnit basic knowledge

SonarQube and JUnit for Software Developers

These are extremely common topics in Java/Spring Boot interviews because most organizations use:

- **JUnit** → Unit Testing
- **SonarQube** → Code Quality Analysis

Part 1: JUnit

What is JUnit?

JUnit is a Java testing framework used to write and execute test cases.

Official project:

JUnit 5

Without JUnit:

```
Java
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
```

You manually test:

```
Java
public static void main(String[] args){
    System.out.println(add(2,3));
}
```

With JUnit:

```
Java
@Test
void testAdd() {
    assertEquals(5, calculator.add(2,3));
}
```

Why JUnit?

Benefits:

- Automated testing
- Regression testing
- Faster development
- Improved code quality
- CI/CD integration

JUnit Annotations

@Test

Marks a test method.

```
Java
@Test
void testAddition() {
}
```

@BeforeEach

Runs before every test.

```
Java
@BeforeEach
void setup() {
    calculator = new Calculator();
}
```

@AfterEach

Runs after every test.

```
Java
@AfterEach
void cleanup() {
}
```

@BeforeAll

Runs once before all tests.

```
Java
@BeforeAll
static void init() {
}
```

@AfterAll

Runs once after all tests.

```
Java
@AfterAll
static void destroy() {
}
```

Assertions

Assertions verify expected results.

assertEquals

```
Java
assertEquals(10, result);
```

assertTrue


```
Java
assertTrue(age > 18);
```

assertFalse

```
Java
assertFalse(list.isEmpty());
```

assertNull

```
Java
assertNull(employee);
```

assertNotNull

```
Java
assertNotNull(employee);
```

assertThrows

Checks exceptions.

```
Java
assertThrows(
    ArithmeticException.class,
    () -> calculator.divide(10,0)
);
```

Interview Question:

Why use assertThrows?

To verify exception handling behavior.

Sample JUnit Test

```
Java
class CalculatorTest {

    Calculator calculator;

    @BeforeEach
    void setup() {
        calculator = new Calculator();
    }

    @Test
```

```
void testAddition() {
    assertEquals(5,
        calculator.add(2,3));
}
```

Unit Testing vs Integration Testing

Unit Test

Tests a single class.

Java
Calculator

No database.

No API.

No external systems.

Integration Test

Tests multiple components together.

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

Interview Question:

Difference between Unit Test and Integration Test?

Unit Test	Integration Test
Tests one component	Tests multiple components
Fast	Slower
No DB	DB may be involved
Isolated	End-to-end flow

Mockito

Very commonly asked with JUnit.

Official site:

Mockito

Used to mock dependencies.

Example:

```
Java
@Service
public class CustomerService {

    @Autowired
    CustomerRepository repo;
}
```

Mock repository:

```
Java
@Mock
CustomerRepository repo;
```

Inject mock:

```
Java
@InjectMocks
CustomerService service;
```

Common Mockito Annotations

@Mock

Creates fake object.

```
Java
@Mock
CustomerRepository repo;
```

@InjectMocks

Injects mocks.

```
Java
@InjectMocks
CustomerService service;
```

when()

Define behavior.

```
Java
when(repo.findById(1L))
    .thenReturn(customer);
```

verify()

Verify method call.

```
Java
verify(repo).save(customer);
```

Interview Question:

Why use Mockito?

To isolate the class being tested from external dependencies.

JUnit Interview Questions

Basic

What is JUnit?

Java testing framework.

Why use JUnit?

To automate testing.

What is a test case?

A method that verifies expected behavior.

What is assertion?

Validation of expected result.

Intermediate

Difference between @BeforeEach and @BeforeAll?

BeforeEach	BeforeAll
Runs before every test	Runs once

Difference between Unit and Integration Testing?

Very common interview question.

Why use Mockito?

To mock external dependencies.

What is test coverage?

Percentage of code executed by tests.

Part 2: SonarQube

What is SonarQube?

SonarQube is a static code analysis tool.

Official site:

[SonarQube](#)

It analyzes:

- Code quality
 - Bugs
 - Vulnerabilities
 - Code smells
 - Test coverage
 - Duplications
-

Why Use SonarQube?

Without SonarQube:

```
Java
public String getName() {
    return null;
}
```

Potential issues remain hidden.

With SonarQube:

```
Bug Detected
Code Smell Detected
Coverage Low
```

SonarQube Architecture

```
Developer
↓
```

```

Git
↓
Jenkins/GitLab
↓
Sonar Scanner
↓
SonarQube Server

```

Important SonarQube Concepts

Code Smell

Not a bug but poor coding practice.

Example:

```

Java
if(flag == true)

```

Better:

```

Java
if(flag)

```

Bug

Potential runtime issue.

Example:

```

Java
String name = null;
name.length();

```

May cause:

```

NullPointerException

```

Vulnerability

Security issue.

Example:

```

Java
SQL Injection
Hardcoded Password

```

Duplication

Same code repeated.

Bad:

Java

```
calculateTax();
calculateTax();
calculateTax();
```

Should move to reusable method.

Technical Debt

Effort needed to improve code quality.

Example:

```
Quick Fix Today
Proper Fix Later
```

Quality Gate

One of the most asked SonarQube interview topics.

Quality Gate is a set of rules.

Example:

```
Coverage > 80%
Bugs = 0
Critical Vulnerabilities = 0
```

Build fails if rules are violated.

Interview Question:

What is a Quality Gate?

A set of code quality conditions that must be met before deployment.

Code Coverage

Measures tested code percentage.

Example:

```
100 Lines
80 Tested

Coverage = 80%
```

Formula:

$$Coverage = \frac{Tested\ Lines}{Total\ Lines} \times 100$$

Example:

$$80/100 \times 100 = 80\%$$

SonarQube Metrics

Common metrics:

Metric	Meaning
Bugs	Potential defects
Vulnerabilities	Security issues
Code Smells	Maintainability issues
Coverage	Test coverage
Duplication	Duplicate code
Technical Debt	Cleanup effort

SonarQube in CI/CD

Flow:

```

Developer
  ↓
Git Push
  ↓
Jenkins Pipeline
  ↓
Build
  ↓
JUnit Tests
  ↓
SonarQube Analysis
  ↓
Quality Gate
  ↓
Deploy
  
```

Interview Question:

What happens if Quality Gate fails?

Deployment can be blocked until issues are fixed.

SonarQube Interview Questions

Basic

What is SonarQube?

Static code analysis tool.

Why use SonarQube?

Improve code quality and security.

What is Code Smell?

Poor coding practice affecting maintainability.

What is Technical Debt?

Future effort required to improve code.

What is Code Coverage?

Percentage of code tested by automated tests.

Intermediate

What is Quality Gate?

A set of quality rules that code must pass.

Difference between Bug and Code Smell?

Bug	Code Smell
Causes incorrect behavior	Poor design/practice
Functional issue	Maintainability issue

What is Static Code Analysis?

Analyzing source code without executing it.

Scenario-Based Questions

SonarQube shows 50% coverage. What will you do?

- Add more JUnit tests

- Cover edge cases
 - Cover exception paths
-

Build failing because of Quality Gate?

Check:

- Bugs
- Vulnerabilities
- Coverage
- Duplicated code

Fix issues and rerun pipeline.

Most Important Topics for Interviews

JUnit

- @Test
- @BeforeEach
- @AfterEach
- Assertions
- assertThrows
- Unit Testing
- Integration Testing
- Mockito
- @Mock
- @InjectMocks
- when()
- verify()

SonarQube

- Code Smells
- Bugs
- Vulnerabilities
- Technical Debt
- Code Coverage
- Static Analysis
- Quality Gate

- Sonar Scanner
- CI/CD Integration

For **2–5 years Java/Spring Boot developers**, the most commonly asked practical questions are:

1. Difference between Unit and Integration Testing.
2. Mockito annotations and usage.
3. How you write JUnit tests for Service classes.
4. What is Code Coverage?
5. What is a SonarQube Quality Gate?
6. Why did your build fail in SonarQube?
7. How do you improve code quality and coverage?